

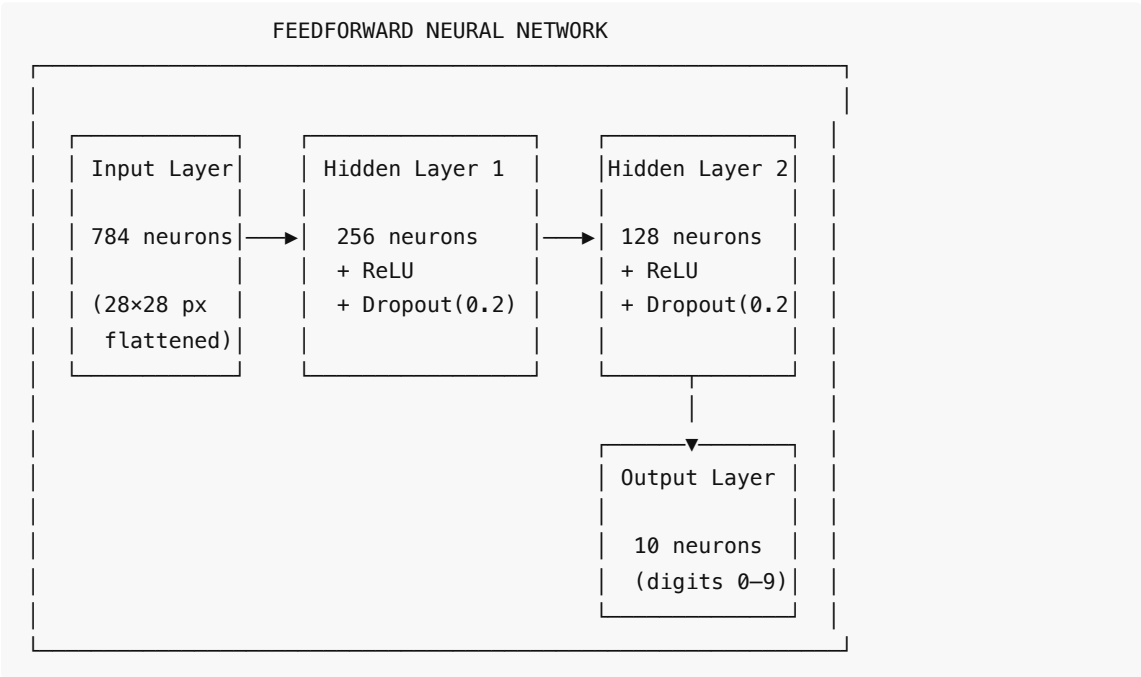
Homework 2: Feedforward Digit Classifier

Course: AI for Augmented and Virtual Reality (CSC 5991)
Author: Fahad Qaseem Khawar - hr4625
Dataset: MNIST — handwritten digits (0–9)
Framework: PyTorch 2.7.1
Device: Apple MPS (Metal Performance Shaders)
GitHub: github.com/fahadqaseem/feedforward-digit-classifier

1. Network Architecture

The model is a **feedforward neural network** (also called a Multi-Layer Perceptron, MLP) with two hidden layers. Information flows in one direction only: from the input layer through the hidden layers to the output layer — no cycles or feedback loops.

Architecture Diagram



Layer Details

Layer	Type	Input Size	Output Size	Parameters
Flatten	—	$1 \times 28 \times 28$	784	0
Linear 1	Fully Connected	784	256	200,960
ReLU 1	Activation	256	256	0
Dropout 1	Regularization (p=0.2)	256	256	0
Linear 2	Fully Connected	256	128	32,896

ReLU 2	Activation	128	128	0
Dropout 2	Regularization (p=0.2)	128	128	0
Linear 3	Fully Connected	128	10	1,290
Total				235,146

Design Choices

- **ReLU activation** ($f(x) = \max(0, x)$): Introduces non-linearity so the network can learn complex patterns. Without it, stacking linear layers is mathematically equivalent to a single linear layer.
- **Dropout (p=0.2)**: During training, randomly deactivates 20% of neurons each forward pass. This prevents the model from relying too heavily on any single neuron, reducing overfitting and improving generalization.
- **No softmax on the output layer**: PyTorch's `CrossEntropyLoss` applies softmax internally, so we output raw logits.

2. Source Code

The full implementation is in [digit_classifier.ipynb](#).

Model Definition

```
import torch.nn as nn

class DigitClassifier(nn.Module):
    """
    Feedforward neural network for MNIST digit classification.

    Architecture:
        Input : 784 (28x28 flattened grayscale image)
        Layer 1: 784 → 256 (Linear + ReLU + Dropout 20%)
        Layer 2: 256 → 128 (Linear + ReLU + Dropout 20%)
        Output : 128 → 10 (Linear, raw logits for each digit class)
    """
    def __init__(self):
        super(DigitClassifier, self).__init__()
        self.network = nn.Sequential(
            nn.Flatten(),                # Reshape [B,1,28,28] → [B,784]
            nn.Linear(784, 256),         # Hidden Layer 1
            nn.ReLU(),
            nn.Dropout(p=0.2),
            nn.Linear(256, 128),         # Hidden Layer 2
            nn.ReLU(),
            nn.Dropout(p=0.2),
            nn.Linear(128, 10)           # Output: 10 classes (digits 0-9)
        )
```

```
def forward(self, x):  
    return self.network(x)
```

Training Loop (core logic)

```
def train_one_epoch(model, loader, criterion, optimizer, device):  
    model.train() # Enable Dropout  
    for images, labels in loader:  
        images, labels = images.to(device), labels.to(device)  
        optimizer.zero_grad() # Clear previous gradients  
        outputs = model(images) # Forward pass  
        loss = criterion(outputs, labels) # Compute loss  
        loss.backward() # Backpropagation  
        optimizer.step() # Update weights
```

3. Training & Testing Information

Dataset

Split	Samples	Description
Training	60,000	Used to update model weights
Test	10,000	Used only for final evaluation

Each image is a **28×28 grayscale** photo of a handwritten digit, flattened to a 784-element vector as input.

Preprocessing:

- `ToTensor()` — converts pixel values from [0, 255] to [0.0, 1.0]
- `Normalize(mean=0.1307, std=0.3081)` — standardizes values using MNIST's dataset-wide statistics so inputs are centered around zero, improving training stability

Hyperparameters

Hyperparameter	Value	Rationale
Batch size	64	Balance between memory use and gradient stability
Learning rate	0.001	Default for Adam; well-suited for this scale
Epochs	10	Sufficient for convergence on MNIST
Optimizer	Adam	Adaptive learning rate; faster convergence than SGD
Loss function	CrossEntropyLoss	Standard for multi-class classification
Dropout rate	0.2	Light regularization; MNIST is not prone to severe overfitting
Random seed	42	Ensures reproducibility

Training Procedure

- 1. For each epoch, iterate over all 938 mini-batches of 64 images
- 2. For each batch: forward pass → compute loss → backpropagation → weight update
- 3. After each epoch, evaluate on the full 10,000-image test set
- 4. Save the model checkpoint whenever test accuracy improves (`best_model.pth`)

Epoch-by-Epoch Training Log

Epoch	Train Loss	Train Acc	Test Loss	Test Acc
1	0.2818	91.31%	0.1139	96.55%
2	0.1305	95.98%	0.0957	97.08%
3	0.0995	97.02%	0.0839	97.29%
4	0.0856	97.41%	0.0776	97.63%
5	0.0738	97.67%	0.0741	97.87%
6	0.0656	97.89%	0.0726	97.75%
7	0.0569	98.21%	0.0728	97.89%
8	0.0550	98.28%	0.0728	97.86%
9	0.0516	98.34%	0.0713	98.09% ← best
10	0.0470	98.40%	0.0680	98.07%

Total training time: 55.4 seconds on Apple MPS (Metal GPU)

4. Test Results

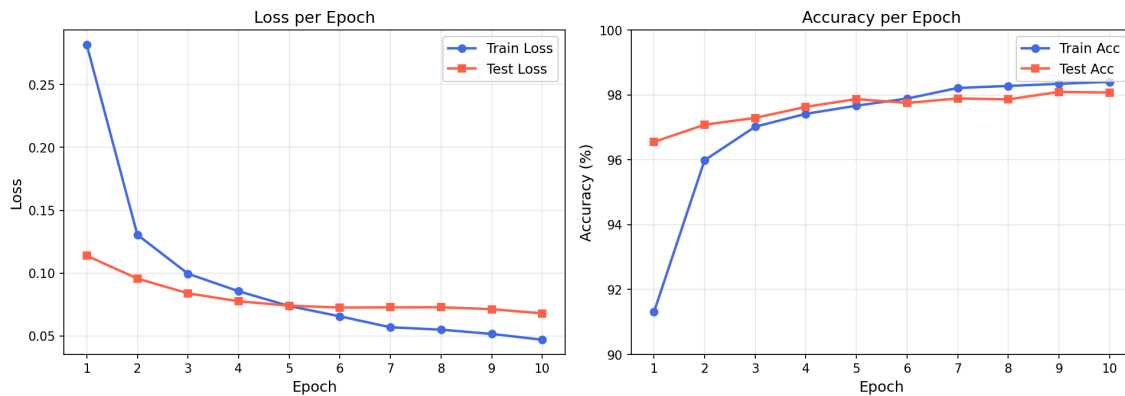
Overall Performance

Metric	Value
Best Test Accuracy	98.09%
Final Test Loss	0.0713
Test Samples Evaluated	10,000
Correctly Classified	9,809 / 10,000

Training Curves

The plot below shows how loss decreased and accuracy increased over 10 epochs for both the training and test sets. The close alignment between the two curves indicates the model is **generalizing well** and is not overfitting.

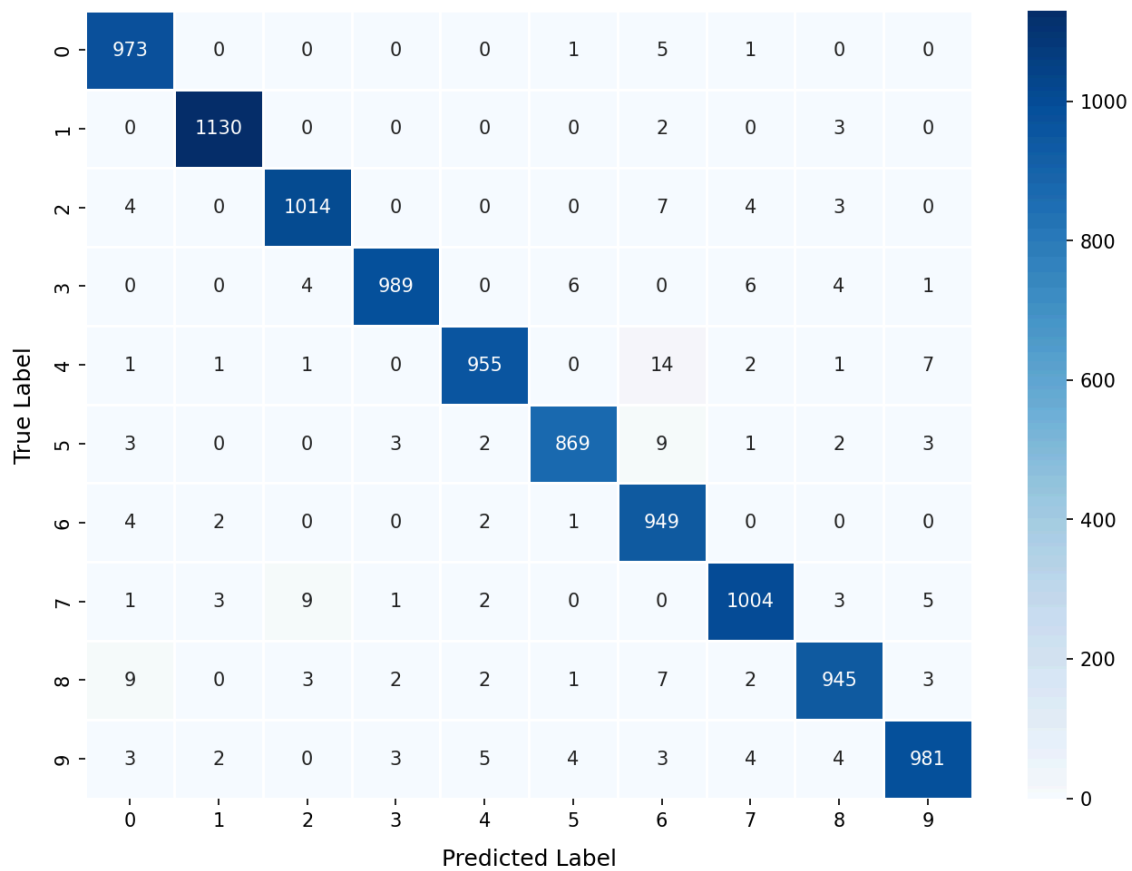
Training History



Confusion Matrix

The confusion matrix shows the model's predictions across all 10 digit classes. The bright diagonal confirms that the vast majority of predictions are correct. The darkest off-diagonal cells reveal that the most common mistakes are between visually similar digits (e.g., 4↔9, 3↔5).

Confusion Matrix on Test Set



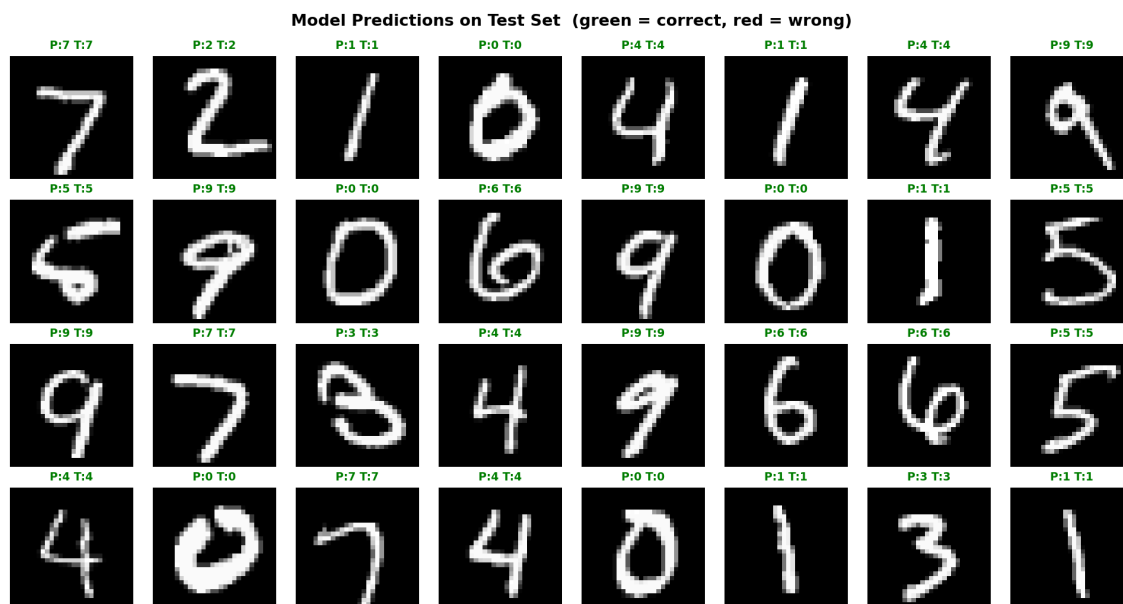
Per-Class Accuracy

Digit	Correct	Total	Accuracy
0	973	980	99.3%
1	1130	1135	99.6%
2	1014	1032	98.3%
3	989	1010	97.9%
4	955	982	97.3%
5	869	892	97.4%
6	949	958	99.1%
7	1004	1028	97.7%
8	945	974	97.0%
9	981	1009	97.2%

- **Easiest:** Digit 1 (99.6%) — visually simple and distinct
- **Hardest:** Digit 8 (97.0%) — can be confused with 3, 5, or 9

Sample Predictions

A grid of 32 test images with predicted (P) vs. true (T) labels. Green = correct, Red = wrong.



Discussion

The model achieved **98.09% test accuracy**, which is well within the expected range for a two-hidden-layer feedforward network on MNIST. Key observations:

1. **Fast convergence:** By epoch 3, the model was already above 97% — most of the learning happens in the early epochs.
 2. **No overfitting:** Train accuracy (98.40%) and test accuracy (98.09%) stay very close throughout training, which means the Dropout regularization worked as intended.
 3. **Balanced performance:** All digit classes achieve above 97% accuracy, showing the model didn't specialize too much on any particular digit.
 4. **Common confusions:** The digits most likely to be misclassified are 8, 9, 4, and 5 — digits that share curved or angular features and look similar when handwritten hastily.
-

Project Files

```
feedforward-digit-classifier/
├─ digit_classifier.ipynb    # Full notebook: model, training, evaluation
├─ best_model.pth           # Saved model weights (best epoch)
├─ requirements.txt         # Python dependencies
├─ README.md               # This report
└─ outputs/
    ├─ training_curves.png   # Loss & accuracy per epoch
    ├─ confusion_matrix.png  # 10×10 prediction heatmap
    ├─ sample_predictions.png # 32 test images with predictions
    ├─ sample_mnist.png      # Sample training images
    └─ class_distribution.png # Dataset class balance chart
```